

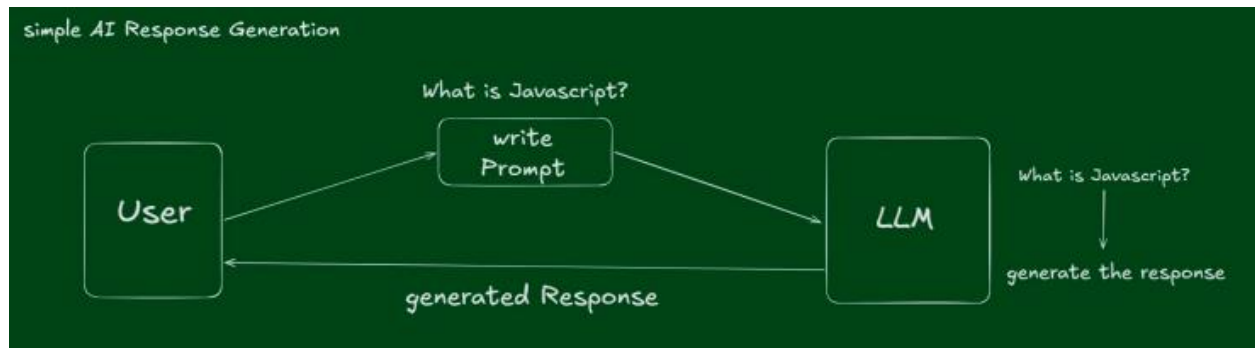
## Advance Backend Part 2

**LLM:** stands for Large Language Model.

It is an AI model that train on massive amount of text data to understand and generate human language.

Example:

openai GPT models, Anthropic Claude, Google Gemini, Meta Llama



Simple ai based Application/website

You ask: "what is ai"

The LLM predicts next tokens and generate response/answer.

**Limitations of LLM:** An LLM by itself:

1. does not access tools automatically.
2. Does not remember long-state reliably.
3. Does not take action in the real world unless connected with system.

It is mainly a text intelligence engine.

### Build simple ai-chatbot using gemini llm api (because its free)

**Level4/** //setup basic express server npm init --- app.listen() etc.

search on google 'google gemini api' & open ' <https://ai.google.dev/gemini-api/docs> '

**flow of chatbot task:** (get gemini api for google ai studio) – ( copy getstarted code mentioned in doc)

We use google gemini sdk for our simple chatbot because SDK give us suggestion while rest api not.

npm install @google/genai

```
import { GoogleGenAI } from "@google/genai";
const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

app.post("/chat", async (req, res) => {
  const { input } = req.body;
  const interaction = await ai.interactions.create({
    model: "gemini-3.5-flash",
    input,
  });
  res.status(200).json({ output_text: interaction.output_text });
});
```

## Example Agent Workflow

User:

"Find the cheapest flight to Delhi and email me details."

Agent:

1. Understands request using LLM
2. Searches flights
3. Compares prices
4. Chooses best option
5. Sends email
6. Returns result

That entire workflow = AI Agent.

LLM + Realtime data + Memory + action

AI Agent

Lang Chain  
Lang Graph

## Difference Between LLM and Agent

| Feature            | LLM     | Agent      |
|--------------------|---------|------------|
| Generates text     | ✓       | ✓          |
| Uses tools/APIs    | ✗       | ✓          |
| Makes decisions    | Limited | ✓          |
| Multi-step tasks   | Weak    | Strong     |
| Memory             | Minimal | Persistent |
| Autonomous actions | ✗       | ✓          |

## Popular Agent Frameworks

Developers commonly use:

- [LangChain ↗](#)
- [LangGraph ↗](#)
- [LlamaIndex ↗](#)
- [CrewAI ↗](#)
- [AutoGen ↗](#)

## What is LangChain?

LangChain is a framework used to build:

- AI chatbots
- RAG systems
- AI agents
- Tool-calling apps
- Multi-step AI workflows

It helps connect:

LLM + Memory + Tools + Vector DB + APIs

## LangGraph

LangGraph is used for:

- Stateful AI agents
- Memory
- Multi-agent systems
- Workflow orchestration

This is production-level AI engineering.

## 1. How to Call LLM in Langchain: <https://docs.langchain.com/oss/javascript/integrations/chat/groq>

Level4/index.js    npm install @langchain/groq @langchain/core    //install langchain pkg for grok

```
import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, temp>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
});

//without langchain CHATBOT
// const ai = new GoogleGenAI({ apiKey: process.env.GOOGLE_API_KEY });

// app.post("/chat", async (req, res) => {
//   const { input } = req.body;
//   const interaction = await ai.interactions.create({
//     model: "gemini-3.5-flash",
//     input,
//   });
//   res.status(200).json({ output_text: interaction.output_text });
// });

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const aiMsg = await llm.invoke(input);
  res.status(200).json({ response: aiMsg.content });
});

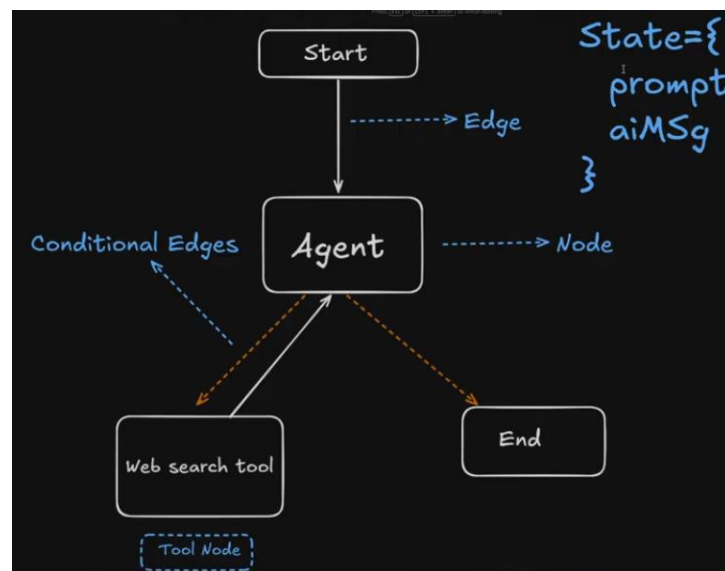
app.listen(port, () => {
  console.log(`Server is Listen on ${port}`);
});
```

**Note:** here we create .env file where we add credentials like GROK\_API\_KEY but not config api in index.js(langchain object), why: because langchain docs says if you write api key with default name 'GROQ\_API\_KEY' then langchain framework auto detect & config it internally, so no need to manually config it.

## 2. Implement AI-Workflow using LangGraph in Code:

### What is LangGraph

langGraph is product of Langchain. It is used to represent ai workflow by graph. Without graph/langGraph write logic of multi-agents ai-workflow is quite difficult.



Agent and Tool both can access State data and langgraph manage/update state.

**Nodes:** Represent specific actions or logic steps (e.g., calling an LLM, querying a database, or running a code snippet). Start & End are built-in nodes, to start and stop ai-workflow.

**Edges:** Determine the control flow or the path taken between nodes based on the output of previous steps.

**Conditional Edges:** Conditional edges are the decision-making components in LangGraph that dynamically route an application's control flow based on the current state. Unlike standard edges which always point to the exact same downstream node, conditional edges execute custom Python logic to determine which path to take next.

**State:** A shared, persistent data structure (like a conversation history) that passes information between nodes throughout the agent's entire execution

### When to Use Which?

>> Use **LangChain** for straightforward tasks like basic chatbots, document parsing, or standard Retrieval-Augmented Generation (RAG) systems.

>> Use **LangGraph** for autonomous AI applications requiring memory, long-running loops, multi-step reasoning, or collaboration between multiple different AI agents.

## Code Implement of AI-Workflow using LangGraph:

npm install @langchain/langgraph @langchain/core // if @langchain/core already installed remove it here because no need to reinstall it.

**flow:** create state/use built-in 'MessagesAnnotation' ----- create graph ----- create function & tools

Level4/index.js

```
import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";
import { Annotation, START, StateGraph } from "@langchain/langgraph";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, tepm>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
});

// implementation of langGraph
// Syntax to create State
const state = Annotation.Root({
  prompt: Annotation,
  aiMsg: Annotation,
});

//create function for graph
const callLLM = async (state) => {
  const response = await llm.invoke([
    {
      role: "system",
      content: "you are ai asistent",
    },
    {
      role: "user",
      content: state.prompt,
    },
  ]),
```

```

]);

    return { aiMsg: response.content };
};

//create graph
const graph = new StateGraph(state)
    .addNode("agent", callLLM)
    .addEdge("__start__", "agent")
    .addEdge("agent", "__end__")
    .compile();

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
    const { input } = req.body;

    const response = await graph.invoke({ prompt: input });
    res.status(200).json({ "ai: ": response });
});

app.listen(port, () => {
    console.log(`Server is Listen on ${port}`);
});

```

## 2.1 Create & Add Tools with LLM to build ai-agent:

Now add tools in llm to expand its functionality and make it ai-agent.

**MessageAnnotation:** langchain ki built-in state hoti h jis ma hm sra jo response h ya input h, osy ik 'messages' array k andr rakhty hn. so is k andr ik messages array hoti h jis k andr sari chzein rakhi hoti hn.

**Tavily Search:** famous web scraping tool. Now we use 'tavily serch langchain'.

[https://docs.langchain.com/oss/javascript/integrations/tools/tavily\\_search](https://docs.langchain.com/oss/javascript/integrations/tools/tavily_search)

npm install @langchain/tavily

create tavily api-key: <https://app.tavily.com/home>

TAVILY\_API\_KEY=lkdjoifewjle

level4/index.js //import & config TavilySearch then add this new tool in tools[] array for tools config.  
Use MessagesAnnotation 'state' which is langgraph built-in state for conditionalEdges.

```

import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";

```

```

import {
  Annotation,
  MessagesAnnotation,
  START,
  StateGraph,
} from "@langchain/langgraph";
import { ToolNode } from "@langchain/langgraph/prebuilt";
import { TavilySearch } from "@langchain/tavily";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

// implementation of langGraph
// create new tool [ tavilyhserch pkg that webserch if llm not have data about
query]
const tool = new TavilySearch({
  maxResults: 5,
  topic: "general",
});

const tools = [tool];
const toolNode = new ToolNode(tools); //runs tools

// setup llm
const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, tepm>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
}).bindTools(tools); //so llm can use tools

// function that cell llm
const callLLM = async (state) => {
  const response = await llm.invoke([
    {
      role: "system",
      content: "you are ai asistent",
    },
    ...state.messages,
  ]);
};

```

```

    return { messages: [response] };
  });

// function for conditionalEdges -agent-shouldcontinue
const shouldContinue = async (state) => {
  const lastMessage = state.messages[state.messages.length - 1];
  if (lastMessage.tool_calls.length > 0) {
    return "tools";
  } else {
    return "__end__";
  }
};

// create graph for ai workflow - using langgraph
const graph = new StateGraph(MessagesAnnotation) //built-in state of langchain
  .addNode("agent", callLLM) // nodename 'agent' jo 'callLLM' function ko call
  kry ga
  .addNode("tools", toolNode)
  .addEdge("__start__", "agent") // edges bnai or start node ko agent node sy
  connect kr dia
  .addEdge("tools", "agent")
  .addConditionalEdges("agent", shouldContinue) //agent node sy 'shouldContinue'
  func run hoga to check kry ga 'tool call' ya 'end node'
  .compile();

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const response = await graph.invoke({
    messages: [{ role: "user", content: input }],
  });
  res
    .status(200)
    // .json({ "ai: ": response.messages[response.messages.length - 1].content
  });
  .json({ response });
});

app.listen(port, () => {
  console.log(`Server is Listen on ${port}`);
});

```



## 2.2 Create & Implement Memory for Our AI Agent

**Stateless agent ( no memory):** Every request is independent; the model sees only the current prompt and forgets everything after responding.

**Stateful agent ( have and manage memory at each state ) :** The agent has a persistent memory/state store (DB, vector DB, Redis, LangGraph state, etc.) where it keeps context, past actions, user preferences, intermediate results, and task progress, and reuses them on the next step or conversation.

### Implementation:

We have 2 options to add memory for llm,

1. Using redis
  2. Using langgraph built-in memory ( MemorySaver from “@langchain/langgraph” )
- for now we use langgraph built-in memory, later in project we use redis for memory.

**Thread Id:** hr ik conversation (new chat) k liay ik id create hoti h jis sy hr ik specific conversation ki memory alg sy save hoti h. so hr ik user & conversation k liay different thread id create hoti h. But for now we hard code single thread id & later in project we learn how to create multiple thread ids.

Level4/Index.js //add MemorySaver() & config it in langgraph & hardcode threadId in graph.invoke

```
import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";
import {
  Annotation,
  MemorySaver,
  MessagesAnnotation,
  START,
  StateGraph,
} from "@langchain/langgraph";
import { ToolNode } from "@langchain/langgraph/prebuilt";
import { TavilySearch } from "@langchain/tavily";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

// implementation of langGraph
// create new tool [ tavilyhserch pkg that webserch if llm not have data about
query]
```

```

const tool = new TavilySearch({
  maxResults: 5,
  topic: "general",
});

const checkPointer = new MemorySaver(); //import {MemorySaver} from
"@langchain/langgraph";

const tools = [tool];
const toolNode = new ToolNode(tools); //runs tools

// setup llm
const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, tepm>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
}).bindTools(tools); //so llm can use tools

// function that call llm
const callLLM = async (state) => {
  const response = await llm.invoke([
    { //note: system -prompt is very important for llm understanding what to do.
      role: "system",
      content: `Your are Jarvis AI assistant, user conversation memory first.

      Only use tools when the answer requires external real-time information
      like: weather, news, web search, stock prices etc.

      Do NOT call tools for simple conversation, memory-based questions,
      greetings, or personal context.`
    },
    ...state.messages,
  ]);

  return { messages: [response] };
};

// function for conditionalEdges -agent-shouldcontinue
const shouldContinue = async (state) => {
  const lastMessage = state.messages[state.messages.length - 1];
  if (lastMessage.tool_calls.length > 0) {
    return "tools";
  } else {

```

```

    return "__end__";
  }
};

// create graph for ai workflow - using langgraph
const graph = new StateGraph(MessagesAnnotation) //built-in state of langchain
  .addNode("agent", callLLM) // nodename 'agent' jo 'callLLM' function ko call
  kry ga
  .addNode("tools", toolNode)
  .addEdge("__start__", "agent") // edges bnai or start node ko agent node sy
  connect kr dia
  .addEdge("tools", "agent")
  .addConditionalEdges("agent", shouldContinue) //agent node sy 'shouldContinue'
  func run hoga to check kry ga 'tool call' ya 'end node'
  .compile({ checkpointer: checkPointer });

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const response = await graph.invoke(
    {
      messages: [{ role: "user", content: input }],
    },
    {
      configurable: { thread_id: "user123" }, //kuxh bhe thread id likh skty ho.
    },
  );
  res
    .status(200)
    .json({ ai: response.messages[response.messages.length - 1].content });
});

app.get("/", (req, res) => {
  res.status(200).json({ message: "helo from server" });
});

app.listen(port, () => {
  console.log(`Server is Listen on ${port}`);
});

```

Note: hm ny koi chezein hawa ma nhe chorien... ya sb implementation documentation ma likhi hoi hn.

<https://docs.langchain.com/oss/javascript/langgraph/checkpointers> is doc pr implemnetation likhi hoi h. apko docs parhny or smjhny any chaheya, doc pr kr onky code dry run kr k dekha krien.

Test ai-agent memory:

```

POST http://localhost:5000/ai
Body
{
  "input": "what is my name?"
}

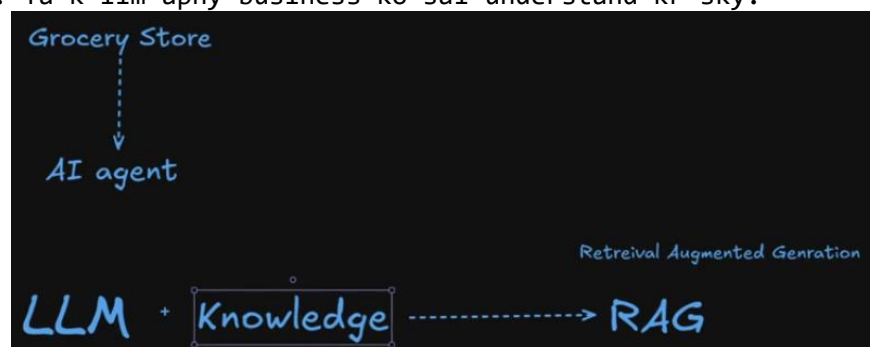
Status: 200 OK Size: 29 Bytes Time: 295 ms
{
  "ai": "Your name is Ayush."
}

```

## ===== RAG ( Retrieval Augmented Generation ) & Vector DB =====

- **Vector Embedding:** words ko number assign krna. Cat → 1.5. ya text ko list of numbers ma convert krna.
- **Vector DB:** save vectors ( save list of no that are assigned to words )

Core idea of RAG: LLM + Add External Knowledge . means hm llm k sth kuxh extra external knowledge add kr dety hn. i.e jsy apny business ki product ki knowledge add kr dein. Ta k llm apny business ko sai understand kr sky.



Matlab AI sirf apni training knowledge use nahi karta.

Wo:

1. Pehle data search karta
2. Relevant information nikalta
3. Fir answer generate karta

## Normal DB vs Vector DB

| Normal DB       | Vector DB         |
|-----------------|-------------------|
| exact text      | semantic meaning  |
| SQL search      | similarity search |
| structured data | embeddings        |
| fast filters    | AI retrieval      |

# RAG Kya Karta

RAG:

PDF read karta  
↓  
Store karta  
↓  
Question aane pe relevant data fetch karta  
↓  
LLM ko deta  
↓  
LLM answer deta

## Vector Database Kya Hota?

Normal DB:

exact match

search karta.

Example:

SELECT \* WHERE name="car"

Vector DB:

similar meaning

search karta.

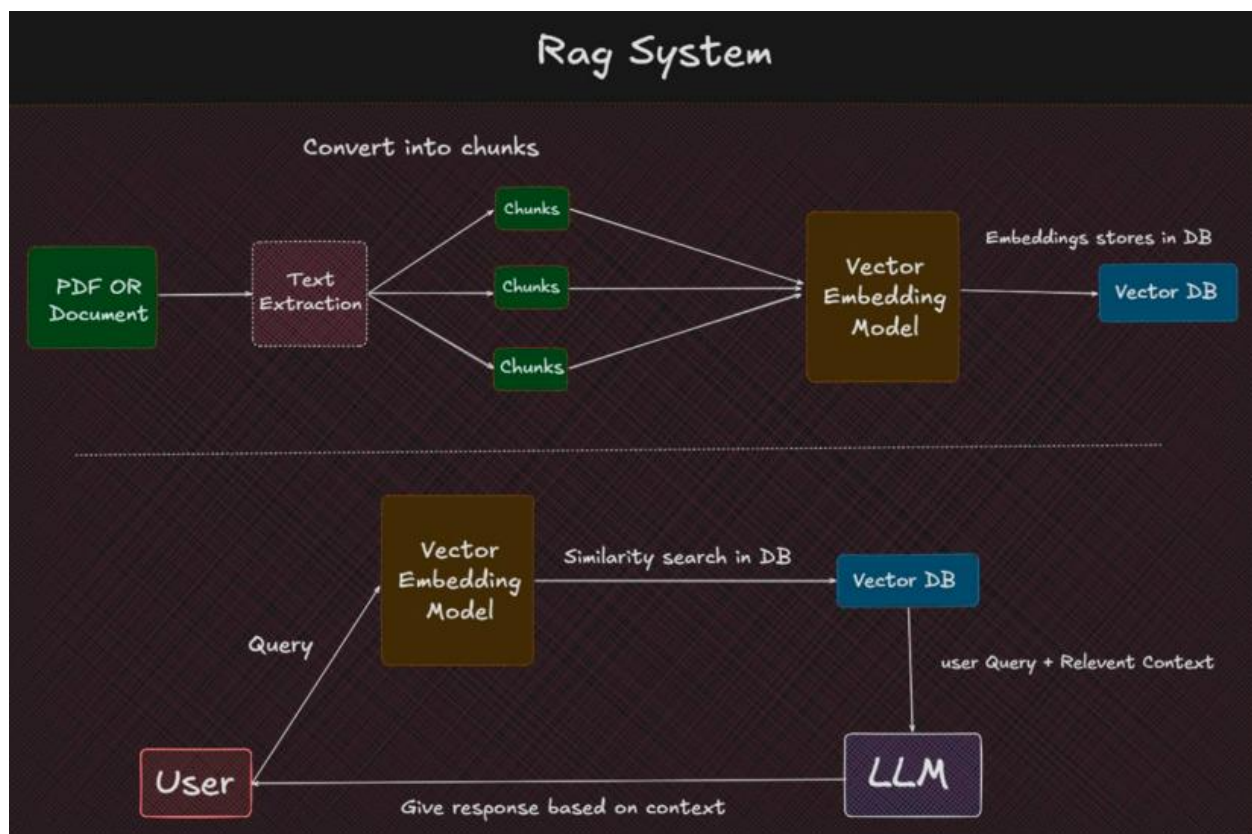
Example:

"vehicle booking"

search karne pe:

- car booking
- auto booking
- cab system

sab mil sakta.

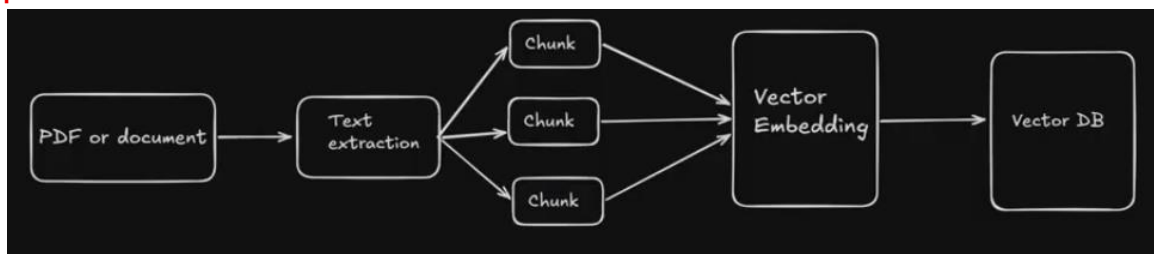


Implementation of RAG System in Code: Level4/phase2/

1. Copy & paste code from level4/phase1 index.js & .env, then remove extra code from index.js and implement simple RAG system.
2. Generate 20 pages pdf of grocery product items category, name, price. 'gen pdf from chatgpt', give this pdf to llm as rag external business data. Level4/phase2/grocery\_store\_items\_list.pdf, you can also upload this pdf through multer that we see in project.

3. Step1 - get pdf done. --- const pdf path & const buffer  
 Step2 - parse pdf & extract text // for this we using 'npm install pdf-parse'.  
 ---- const pdfResult  
  
 step3 - text chunking // npm install @langchain/textsplitters  
 ---- const splitter  
  
 step4 - vector embedding //google vector embedding model langchain  
 "vector embedding models allows you to convert text into multi-dimensional numerical vectors(list). This captures semantic meaning for applications like Retrieval-Augmented Generation (RAG) and semantic search"  
[https://docs.langchain.com/oss/python/integrations/embeddings/google\\_generative\\_ai](https://docs.langchain.com/oss/python/integrations/embeddings/google_generative_ai)  
 npm install @langchain/google-genai @langchain/core @google/generative-ai  
 ---- const embeddings  
  
 step5 - vector DB // Qdrant vector database langchain  
<https://docs.langchain.com/oss/javascript/integrations/vectorstores/qdrant>  
 npm install @langchain/qdrant @langchain/core @langchain/openai  
 ---- const vectorStore  
 after all configuration, run once upload function then goto qdrant-cluster-manageCluster and see new embedding added in cluster.then comment upload function so don't reupload same embeddings.

### RAG Implementation Phase1 start:



Flow: upload func (getpdf, buffer, parse pdf) - embeddings - vectorStore

```

import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";
import { PDFParse } from "pdf-parse";
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";
import fs from "fs";
import { GoogleGenerativeAIEmbeddings } from "@langchain/google-genai";
import { TaskType } from "@google/generative-ai";
import { QdrantVectorStore } from "@langchain/qdrant";

dotenv.config();

const app = express();

```

```

app.use(express.json());
const port = process.env.PORT || 4000;

const embeddings = new GoogleGenerativeAIEmbeddings({
  model: "gemini-embedding-001", // 768 dimensions
  taskType: TaskType.RETRIEVAL_DOCUMENT,
  title: "Document title",
});

const vectorStore = await QdrantVectorStore.fromExistingCollection(embeddings, {
  url: process.env.QDRANT_URL,
  collectionName: "grocery-store",
});

const upload = async () => {
  const pdfPath = "./grocery_store_items.pdf";
  const buffer = fs.readFileSync(pdfPath); //return buffer //Buffer is a global,
  //built-in class used to directly handle and manipulate raw binary data
  const pdfResult = new PDFParse({ data: buffer }); //Initialize the parser with
  //the buffer
  const result = await pdfResult.getText(); //Extract the text from each page
  //req-page.
  const text = result.text; // extract whole text only- no chunking for now.
  //then perform
  const splitter = new RecursiveCharacterTextSplitter({
    chunkSize: 500,
    chunkOverlap: 0,
  });
  const splitted_docs = await splitter.createDocuments([text]);
  await vectorStore.addDocuments(splitted_docs);
};

// upload(); run once to save embeddings on qdrant db cluster

// setup llm
const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, temp>0 -
  //creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
});

// with langchain CHATBOT
app.post("/chat", async (req, res) => {

```

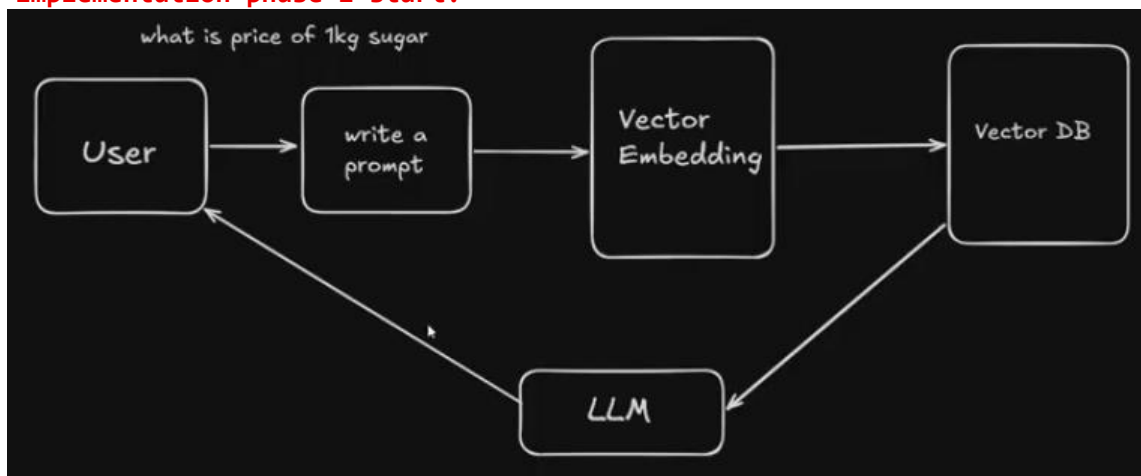
```
const { input } = req.body;

const response = await llm.invoke(input);
res.status(200).json({ ai: response.content });
});
```

.env

```
GOOGLE_API_KEY=AQ.Ab8Rfdgfdgfdgfdvhjrr3vw3PD-18hAVmoH_5g
GROQ_API_KEY=gsk_yTtr9fdgfd3IVFzIutB2gS5swikrz
TAVILY_API_KEY=tvly-dev-fdsgvgfdsgfdgfd
PORT=4000
QDRANT_URL="https://sfdggregsgrrg9afb6.us-east-2-0.aws.cloud.qdrant.io"
QDRANT_API_KEY="eyJhbGciOi0retwretrIsInR5cCI6IkpXVCJ9. "
```

### RAG implementation phase 2 start:



Level4/phase2/index.js //implement RAG phase 2 in /chat route

```
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const docs = await vectorStore.similaritySearch(input, 5);
  const context = docs.map((d) => d.pageContent).join("/n");

  const response = await llm.invoke([
    new SystemMessage(`You are RAG AI asistent.
    STRICT RULES:
    - Answer only from context
    - Do not use outside knowledge
    - If answer not found say: "i don't know from uploaded PDF.
    context: ${context}`),
    new HumanMessage(input),
  ]);
  res.status(200).json({ ai: response.content });
});
```



Note: Now you're capable to make any ai-agents because you leant, langchain, langgraph, RAG, Vector DB.

Follow these steps to build your ai-agent:

1. Think & design workflow first. ( draw workflow)
2. Help ai to implement this workflow in code.

## ===== AWS Cloud Deployment=====

### 1. Cloud Kya Hota Hai

Cloud basically kisi aur company ke powerful servers hote hain jo internet par available hote hain.

Instead of:

- apna physical server kharidna
- electricity
- cooling
- maintenance
- networking

Tum rent par server use karte ho.

Major cloud providers:

- Amazon Web Services
- Google Cloud
- Microsoft Azure

### AWS Introduction

#### AWS Kya Hai

Amazon Web Services duniya ka sabse popular cloud provider hai.

AWS services bahut saari hoti hain, but deployment ke liye initially sirf kuch important services samjho.

Important AWS Services:

1. IAM //control access
2. S3 //database
3. EC2 //virtual server for deploying websites.

#### **flow to deploy project:**

push code on github - create EC2 instance -  
create & assign elastic ip to instance - connect server to local cmd  
through SSH key-pair & connect cmd - pull repo to server - install docker  
& create and run docker container - add port to instance security group -  
access server on <http://elastic-ip/port> and all done.

EC2 - instances - launch instances - server\_name - QuickStart (Ubuntu ) -  
instanceType - create-key-pair-for-login & save it in localDist  
c:/users/pc - allow SSH traffic from 0.0.0.0/0 //so we control server  
from our local pc. --- launch Instance

EC2 - **elastic Ips** - allocate elastic ip. // create elastic ip  
EC2 - elastic Ips - Actions - Associate Elastic IP address - choose a  
instance // here you assign elastic ip to our EC2 virtual server.

Now EC2/our virtual server machine is ready so connect it with our local  
terminal and start deploying server.

EC2 - instances (select instance) - connect - SSH Client - copy example  
ssh cmd.  
local system - open cmd(where key-file stored i.e in our case c:/users/pc)  
- run SSH cmd that mentioned on SSH client page. After you connected with  
virtual server.

> now pull project that already on github to server.  
git clone repo-url                      ls                      cd repo-dir

> now run project in virtual server using same cmds as we run dockerfile  
project on our local system.  
install docker in ubuntu //see doc & install docker on server root then.  
cd level5  
sudo docker build -t level5 . //create docker-image  
Sudo docker run -it -d -p 5000:5000 level5 // run container in background.

➤ How to access it from local cmd: using server elastic ip  
In local browser: <http://elastic-ip/> //first set port in aws.console  
EC2 - network & security - security group - open 1<sup>st</sup> group - edit inbound  
rules - add rule : custom TCP TCP 5000 0.0.0.0/0 --- save.

now it perfectly run on elastic-ip:port                      perfectly.

===== CI/CD using GitHub actions =====

If we make change in code.

push updated code on github - git pull repo on server - old container stop  
-- again rebuild docker image and run container.

its time consuming and slow so CI/CD pipeline solve this problem and  
automate this task. Note: stop instances if not used because it cost us.

## 1. CI/CD Kya Hota Hai

### CI Continuous Integration

Meaning:

code continuously integrate karna

Developer:

- code push kare
- automatically:
  - build
  - test
  - validation

run ho.

## CD

### Continuous Deployment / Delivery

Meaning:

automatic deployment

After successful build:

- server update
- docker rebuild
- ECS deploy

Automatically ho jaye.

Traditional deployment:

```
ssh server
git pull
npm install
restart app
```

Problems:

- downtime
- mistakes
- forgot restart
- wrong branch
- inconsistent deployments

## CI/CD Solves

- ✓ Automation
- ✓ Faster deployment
- ✓ Reliable deployment
- ✓ Team collaboration
- ✓ Rollback support
- ✓ Production consistency

## CI/CD Folder

Most important:

.github/workflows/

Inside:

deploy.yml

```
name: Deploy To EC2

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Connect EC2 and Deploy
        uses: appleboy/ssh-action@v1.0.3

        with:
          host: ${ secrets.HOST }
          username: ubuntu
          key: ${ secrets.SSH_KEY }

      script: |
        cd /home/ubuntu/project

        git pull origin main

        docker compose down

        docker compose up --build -d
```

4. ECR & ECS (AWS services ): push docker-images on ECR and run container on ECS through ECR-image  
... its notes pending

Course End